

1/2/26
1) What is Express.js?

Express.js is a lightweight and fast web framework for Node.js. It helps us build web applications and REST APIs easily by providing features like routing, middleware, request handling, and response handling.

2) Why do we use Express instead of the HTTP module?

HTTP Module

- * Build into Node.js
- * Everything must be written manually
- * More code

Express.js

- * Build on top of the HTTP module
- * Less code.
- * Easy routing
- * Middleware support

3) What is Middleware?

Middleware is a function that executes b/w the request and the response.

- * Read the request
- * Modify the request
- * Validate data
- * Authenticate users

* Log information

* End the request

* Pass control to the next middleware

4) D/B API and Endpoint?

API:

API stands for Application Programming Interface. It is a collection of multiple endpoints.

Contains

GET /users

POST /users

PUT /users/:id

DELETE /users/:id

EndPoint:

An endpoint is a single URL where a specific request is handled.

Example

GET /users is one endpoint

5) What is Route Mounting?

Route mounting means connecting routes from separate files to the main Express application.

authRoutes.js

```
router.post("/register", register);
```

server.js

```
app.use("/api/auth", authRoutes);
```

Actual URL becomes

/api/auth/register

6) What is req.body?

req.body contains data sent by the client inside the request body.

7) D/B req.params and req.query?

req.params

Used for URL parameters

req.query

Used for query strings

8) What is Mongoose?

Mongoose is an ODM (Object Data Modeling) library for MongoDB in Node.js

It helps:

- * Connect MongoDB

- * Create schema

- * Create Models

- * Validate data

- * Perform CRUD operations

9) D/B Schema and Model?

Schema:

Schema defines the structure of a document.

Model:

A model is created using the Schema.

The model is used to perform database operations.

```
user.find();  
user.create();  
user.updateOne();  
user.deleteOne();
```

10) Why do we use bcrypt during registration and login?

Passwords should never be stored as plain text because if the database is compromised, attackers can read all user passwords.

bcrypt protects passwords by converting them into a secure hash.

Login:

When the user logs in, compare the entered password with the stored hashed password.

If isMatch is true, the password is correct:

```
const isMatch = await bcrypt.compare(password, user.password);
```

bCrypt:

- * Encrypts passwords using hashing (one-way hashing).

- * Protects user data even if the database is leaked.

- * Adds a salt automatically to make identical passwords produce different hashes.

- * Makes brute force attacks much harder.